BLDC六步换向UVW相规则通道电流采样，二维数组，指针函数，ADC与DMA，分析运算电路放大倍数

正点原子BLDC六步换向

使用ADC规则通道连续转运，扫描模式，一次5个通道，配合DMA传输，连续转运50次之后，一共250个数据，在DMA中断里面进行均值滤波。

在DMA中断内进行均值滤波的时候，需要把ADC CCR寄存器与DMA CR寄存器关闭，也就是说需要把ADC与DMA的使能位全部关闭，否则进入中断ADC DR寄存器任然会有值在更新，那么此时如果关闭DMA，就会造成ADC DR寄存器数据溢出，导致出现溢出标志位，无法再滤波函数（本质是for循环）之后再开启ADC的使能位。

因此正点原子在使用HAL库的时候有用到DMA中断，在中断里面进入DMA中断服务函数，在DMA中断函数里面有判断标准位与失能DMA，再进入到ADC回调函数（DMA中断都会进入ADC回调函数），再在ADC回调函数中，使用函数HAL_ADC_Stop_DMA(&g_adc_nch_dma_handle); /* 关闭DMA转换 */ 这个函数会同时关闭ADC与DMA使能位，这样就不会造成溢出，同时伴随是否在工作状态的判断，再进行滤波函数的操作，这样就不会造成同时使用dma获取到的ADC数组情况。

再在中断结束前开启ADC 与DMA这样保持正常使用。

HAL_ADC_Start_DMA(&g_adc_nch_dma_handle, (uint32_t *)&g_adc_value, (uint32_t)(ADC_SUM)); / 再启动DMA转换*/

/**

- @brief ADC DMA采集中断服务函数
- @param 无
- @retval 无

*/

void ADC_ADCX_DMASx_IRQHandler(void)

{

HAL_DMA_IRQHandler(&g_dma_nch_adc_handle);

}

uint16_t g_adc_val[ADC_CH_NUM]; /* ADC平均值存放数组 */

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)

{

if (hadc->Instance == ADC1) /* 大约2.6ms采集完成进入中断 /

{

HAL_ADC_Stop_DMA(&g_adc_nch_dma_handle); / 关闭DMA转换 /

calc_adc_val(g_adc_val); / ADC数值转换 */

HAL_ADC_Start_DMA(&g_adc_nch_dma_handle, (uint32_t *)&g_adc_value, (uint32_t)(ADC_SUM)); / 再启动DMA转换*/

}

}

遇到问题

在本文作者移植到标准库的过程中，一开始采用ADC规则通道的中断，但是ADC规则通触发是连续转运模式，并不同FOC一般，用定时器触发，因此，规则通道只要完成转换就会进入中断服务函数，但是此时ADC与DMA并没有关闭，因此ADC与DMA一直在进行操作。ADC中断服务函数中关闭ADC的使能，就会造成无法开启的操作。

后续换成DMA中断，仍然是这个问题，如果选择DMA循环模式，ADC DR寄存器的值一直在更新，那么在进入DMA中断函数的时候，DMA其实一直在工作NDTR寄存器的值一直在改变，因此在DMA内进行均值滤波的话，会同时对这个数组进行调用（目前不影响取值，暂时采用本方法）

如果换成DMA循环模式，在DMA中断关闭DMA使能位，再开启，且重新给NDTR寄存器计次幅值，会导致ADC DR没关闭，因此ADC CCR会报溢出错误，无法重新使能ADC工作（强行使能ADC EN位也无法使能）

如果换成循环模式，出现问题扔就是只能进入一次DMA中断，无法第二次进入，本质上就是ADC无法重新使能，目前未找到解决方法，需要查看HAL库如何处理

目前配置

/**

- @brief　　DMA中断配置
- @param　　无
- @retval
- 有BUG，暂不影响操作，在进入DMA中断时,ADC转运仍在进行，在DMA中断里面无法关闭ADC与DMA
  */

void DMA2_Stream4_IRQHandler (void)
{
  if(DMA_GetFlagStatus(DMA2_Stream4, DMA_FLAG_TCIF4) != RESET)
  {
    mid_ADC1_Average_Value(g_adc_val, g_adc_value);
    /* 清除标志位，高低位都需要清除 */
    DMA_ClearITPendingBit(DMA2_Stream4,DMA_FLAG_TCIF4);
    DMA_ClearITPendingBit(DMA2_Stream4,DMA_FLAG_HTIF4);

```
  }
```

}

DMA的配置

DMA中断中，有许多标志位，记得统一清理全部传输完成，半传输完成等

/**

- @brief　　DMA中断配置
- @param　　无

- @retval 无
- /
  static void bsp_DMA_NVIC(void)
  {
  NVIC_InitTypeDef NVIC_InitStructure;
  / NVIC中断服务配置 /
  NVIC_InitStructure.NVIC_IRQChannel =DMA2_Stream4_IRQn;
  NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
  NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority =1;
  NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
  NVIC_Init(&NVIC_InitStructure);
  / NVIC中断服务配置,DMA中断 */
  DMA_ClearITPendingBit(DMA2_Stream4, DMA_IT_TC);

}

/**

- @brief DMA初始化配置
- @param 配置参数开启DMA流，外设地址，DMA地址，缓存区存储大小
- @arg 需要配置的DMA数据流 DMA_Stream_TypeDef类型
- @arg 需要配置的DMA通道
- @arg 外设基地址
- @arg 存储区地址
- @arg 存储区数据大小
- @retval 无
- /
  *static void bsp_DMA_Config(DMA_Stream_TypeDef*DMA_Streamx,uint32_t chx, uint32_t par, uint32_t mar , uint16_t ndtr)
  {
  DMA_InitTypeDef DMA_InitStruct;
  /* 开启对应DMA数据流时钟 /
  if((uint32_t)DMA_Streamx>(uint32_t)DMA2)
  {
  RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2,ENABLE);
  }
  else
  {
  RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA1,ENABLE);
  }
  DMA_DeInit(DMA_Streamx);
  while (DMA_GetCmdStatus(DMA_Streamx) != DISABLE){}

  / 外设基地址 /
  DMA_InitStruct.DMA_PeripheralBaseAddr = par;
  / DMA存储器基地址 /
  DMA_InitStruct.DMA_Memory0BaseAddr = mar;
  / 数据传输方向 /
  DMA_InitStruct.DMA_DIR = DMA_DIR_PeripheralToMemory;

```
    / 缓存区存储大小 /
    DMA_InitStruct.DMA_BufferSize = ndtr;
    / 外设地址不需要自增 /
    DMA_InitStruct.DMA_PeripheralInc =DMA_PeripheralInc_Disable;
    / 存储器地址自增 /
    DMA_InitStruct.DMA_MemoryInc = DMA_MemoryInc_Enable;
    / 外设数据和存储器数据都选半字16位 /
    DMA_InitStruct.DMA_MemoryDataSize = DMA_PeripheralDataSize_HalfWord;
    DMA_InitStruct.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
    / 模式为循环模式 /
    DMA_InitStruct.DMA_Mode = DMA_Mode_Circular;
    / 优先级为高优先级 /
    DMA_InitStruct.DMA_Priority = DMA_Priority_Medium;
    / 禁用FIFO模式 /
    DMA_InitStruct.DMA_FIFOMode = DMA_FIFOMode_Disable;
    / 以下参数都是FIFO的参数，不使用可以不配置 /
    DMA_InitStruct.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;
    DMA_InitStruct.DMA_MemoryBurst = DMA_MemoryBurst_Single;
    DMA_InitStruct.DMA_PeripheralBurst =DMA_PeripheralBurst_Single;
    / DMA通道 */
    DMA_InitStruct.DMA_Channel =  chx;
    bsp_DMA_NVIC();
    DMA_Init(DMA_Streamx,&DMA_InitStruct);
    DMA_Cmd(DMA_Streamx,ENABLE);
    DMA_ITConfig(DMA_Streamx, DMA_IT_TC, ENABLE);
     while(DMA_GetCmdStatus(DMA_Streamx) != ENABLE)
    {
    }
}
```

ADC配置

目前来说配置规则通道中断要注意EOC标准位

```
/**
  @brief  ADC规则通道参数设置
  @param
  @retval
  */
static void bsp_ADCRegularChannelConfiguration(void)
{
#define     SampleTime_Regular      ADC_SampleTime_56Cycles
ADC_RegularChannelConfig(ADC_ADCX, ADC_ADCX_CH0, 1, SampleTime_Regular);
ADC_RegularChannelConfig(ADC_ADCX, ADC_ADCX_CH1, 2, SampleTime_Regular);
ADC_RegularChannelConfig(ADC_ADCX, ADC_ADCX_CH2, 3, SampleTime_Regular);
ADC_RegularChannelConfig(ADC_ADCX, ADC_ADCX_CH3, 4, SampleTime_Regular);
ADC_RegularChannelConfig(ADC_ADCX, ADC_ADCX_CH4, 5, SampleTime_Regular);
}
```

*///***
*// * @brief ADC1中断配置*
*// * @param 无*
*// * @retval 无*
*// **
// /
//static void bsp_ADC1_NVIC(void)
//{
// NVIC_InitTypeDef NVIC_InitStructure;
// ADC_ClearFlag(ADC1, ADC_FLAG_EOC);
// / NVIC中断服务配置 /
// NVIC_InitStructure.NVIC_IRQChannel =ADC_IRQn;
// NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
// NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority =2;
// NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
// NVIC_Init(&NVIC_InitStructure);
*// / NVIC中断服务配置,需要配置的中断是EOC规则组中断 */*
// ADC_ITConfig(ADC1, ADC_IT_EOC, ENABLE);
//}

*/***

- *@brief ADC1初始化配置*
- *@param 无*
- *@retval 无*
- */*
  static void bsp_ADC1_Config(void)
  {
  ADC_CommonInitTypeDef ADC_CommonInitStructure;
  ADC_InitTypeDef ADC_InitStructure;
  */ 复位ADC时钟 */*
  ADC_DeInit();

```
/* 开启ADC1时钟 */
ADC_ADCX_CLK_ENABLE();
```

```
ADC_CommonStructInit(&ADC_CommonInitStructure);
ADC_CommonInitStructure.ADC_DMAAccessMode=ADC_DMAAccessMode_Disabled;
ADC_CommonInitStructure.ADC_Mode=ADC_Mode_Independent;
/* 四分频 84/4=21Mhz */
ADC_CommonInitStructure.ADC_Prescaler=ADC_Prescaler_Div4;
ADC_CommonInitStructure.ADC_TwoSamplingDelay=ADC_TwoSamplingDelay_5Cycles;
ADC_CommonInit(&ADC_CommonInitStructure);

ADC_StructInit(&ADC_InitStructure);
ADC_InitStructure.ADC_ContinuousConvMode=ENABLE;
ADC_InitStructure.ADC_DataAlign=ADC_DataAlign_Right;
ADC_InitStructure.ADC_ExternalTrigConv=ADC_ExternalTrigConvEdge_None;
ADC_InitStructure.ADC_NbrOfConversion=5;
```

```
ADC_InitStructure.ADC_Resolution=ADC_Resolution_12b;
ADC_InitStructure.ADC_ScanConvMode=ENABLE;
ADC_Init(ADC_ADCX, &ADC_InitStructure);
/* 配置规则通道 */
bsp_ADCRegularChannelConfiguration();
/* 开启DMA连续转运 */
ADC_DMACmd(ADC_ADCX, ENABLE);
ADC_DMARequestAfterLastTransferCmd(ADC_ADCX, ENABLE);
ADC_Cmd(ADC_ADCX,ENABLE);
```

}

/**

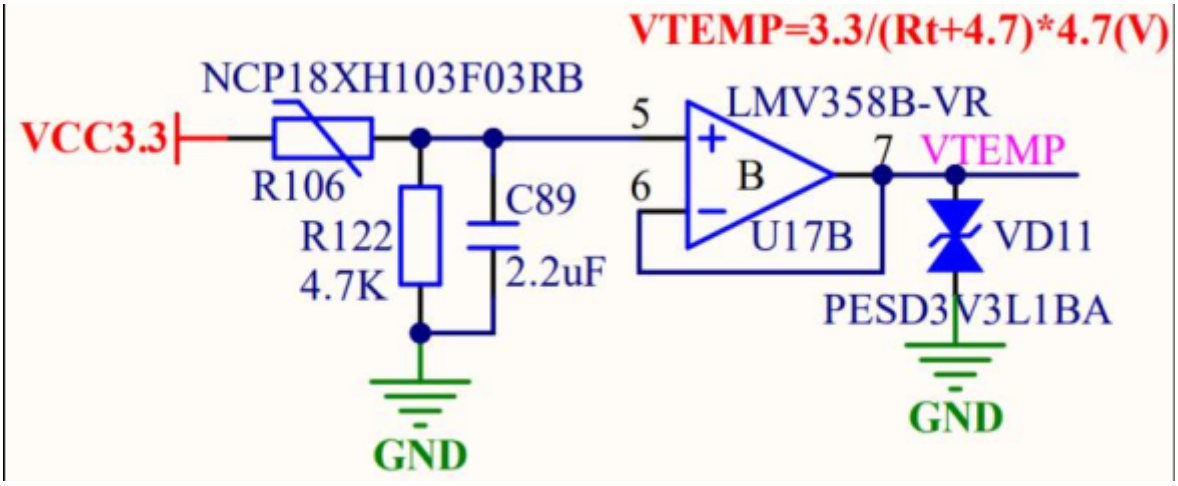
- @brief　　ADC初始化
- @param　　无
- @retval　　无
- */
  void bsp_ADC1_Init(void)
  {
  bsp_ADC1Gpio_Config();
  bsp_ADC1_Config();

}

ADC采样完成后对数据处理

温度传感器电路分析

放大倍数 K=1+Rf/R1-Rf/R2



如图，温度传感器电路

首先分析运算放大器

由于虚短与虚断Vin = Vout 因此这是一个跟随器

又有NTC电阻与R122的4.7K电阻分压，那么VTEMP = 3.3V/(Rt+4.7K) *4.7K

VTEMP= ADC1_CH0 * 3.3/4096

则Rt = (4096*4.7K)/CH0 - 4.7K

```
float mid_Get_Temperature(uint16_t para)
{
    float Rt;
    float temp;
    Rt = 3.3f * 4700.0f / (para * 3.3f / 4096.0f) - 4700.0f;
```

根据手册获得公式

T1=1 / (ln (Rt/R0) / B + 1/T2)

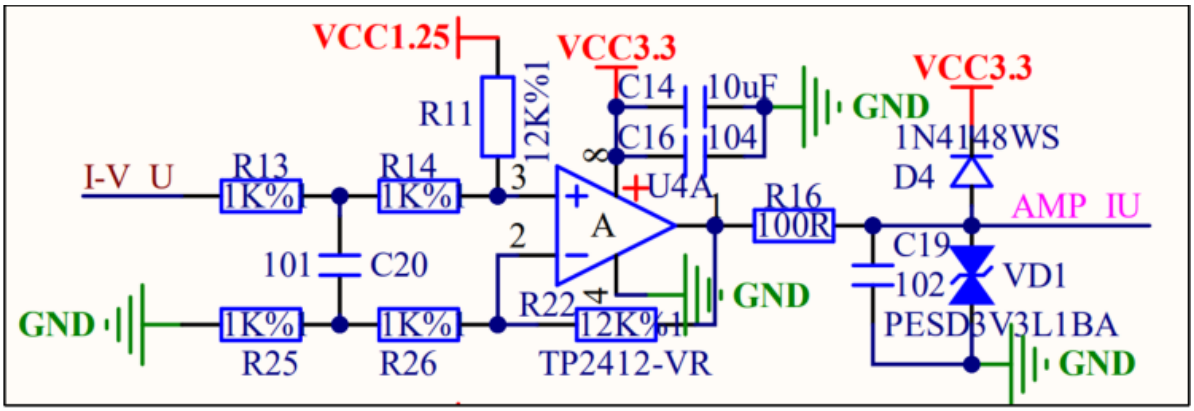
具体参数都使用宏定义提前定义

这里的 T1 和 T2 指的是 K 度即开尔文温度，K 度 = 273.15(绝对温度

- T2 ： 273.15 + 25（25 摄氏度下的 K 度值）
- T1 ： 实际温度
- Rt ： 热敏电阻在 T1 温度下的阻值；
- R0 ： 热敏电阻在 T2 常温下的标称阻值；
- B 值： 热敏电阻的重要恒定参数；

```
Rt = 3.3f * 4700.0f / (para * 3.3f / 4096.0f) - 4700.0f;
/* like this R=5000, T2=273.15+25,B=3470, RT=5000*EXP(3470*(1/T1-1/(273.15+25)) */
temp = Rt / Rp;
/* ln(Rt/Rp) */
temp = log(temp);
temp /= Bx;
/* ln(Rt/Rp)/B */
temp += (1.0f / T2);
temp = 1.0f / (temp);
temp -= Ka;
return temp;
```

电流采集分析



首先分析运算放大器

使用虚短 虚断分析

虚断分析电路，V+端属于断路，那么该电路电流可以表示

V1.25-Vin/12K = V+ - Vin/1K+1k (I-V 电压最大0.2V)

虚短分析电路

V+ = V- 端

V+/1K+1K = Vout-V+ /12K

联立方程求解

Vout = 6Vin + 1.25V

由于AMP IU 是单片机采集的CH1通道

CH1 * 3.3/4096 = 6*(0.02* I)+1.25V

因为上电的时候会求零漂也就是把1.25V电压求出模拟量OFFSET后又减去，因此

转换统一单位mA

```
/* ADC采集值 * 3.3/4.096 （mv） = 6 * ( 0.02*I ) */
#define ADC_CURRENT    (float)(3.3f / 4.096f / 0.12f)
```

I=CH1 * 3.3V/4096 *0.12

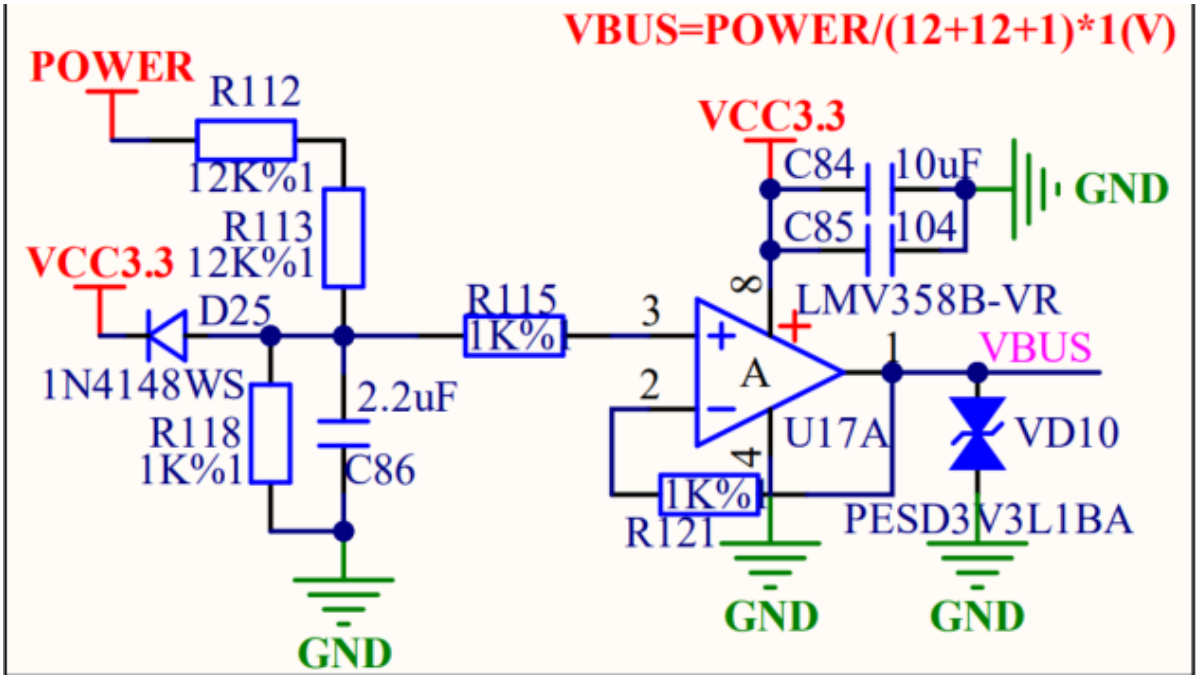
上电求零漂，上电开启TIM1，当电机停止是运行这个函数，这个中断一直会开启

```
void TIM7_IRQHandler(void)
{
    if (TIM_GetITStatus(TIM7, TIM_IT_Update) != RESET)  /* 检查中断标志位 */
        {
            if(g_bldc_motor1.run_flag == 0)
            {
                mid_Get_Current_Offset();
            }

            TIM_ClearITPendingBit(TIM7, TIM_IT_Update  );  /* 清除中断标志位 */
            //LED1_TOGGLE();

        }
}
```

求零漂构建二维数组，求均值，在建立二维数组的时候多创建一组，可以存在数组012后面最后一层

```
void mid_Get_Current_Offset(void)
{
    uint16_t i;
    uint32_t avg[3]={0};
    g_motor_data.adc_amp_offset[0][g_motor_data.adc_amp_offset_p] = g_adc_val[2];
    g_motor_data.adc_amp_offset[1][g_motor_data.adc_amp_offset_p] = g_adc_val[3];
    g_motor_data.adc_amp_offset[2][g_motor_data.adc_amp_offset_p] = g_adc_val[4];
    g_motor_data.adc_amp_offset_p++;
    NUM_CLEAR(g_motor_data.adc_amp_offset_p,ADC_COLL);
    for(i=0 ; i<ADC_COLL ;i++)
    {
        avg[0] += g_motor_data.adc_amp_offset[0][i];
        avg[1] += g_motor_data.adc_amp_offset[1][i];
        avg[2] += g_motor_data.adc_amp_offset[2][i];
    }
    for(i=0 ; i<3 ; i++)
    {
        avg[i]/=ADC_COLL;
        g_motor_data.adc_amp_offset[i][ADC_COLL] =avg[i] ;
    }
```

电源电压采集分析



运用虚断与虚断分析电路V

虚断

Vin=V+

I+= Vin/1K

虚短

I- = Vout/1K

I+=I-

Vin=Vout

又Vin = POWER * 1K/(12K+12K+1K)

那么算出

Power= CH2 * 3.3/4096 * (12K+12K+1K)

```
#define ADC_SUPPLY        (float)(3.3f * 25 / 4096)          /* ADC采集值 * 3.3/4096 （V）= POW
```

使用宏定义完成低通滤波

```
/*一阶数字滤波*/
/* Yn:out;Xn:in;a:系数 */
#define FirstOrderRC_LPF(Yn_1,Xn,a) Yn_1 = (1-a)*Yn_1 + a*Xn;
/*****************************************************************
```

算母线总电流，就算UVW其中两相导通上面电流多少

if(g_bldc_motor1.step_sta == 0x05)

{

/* UV */

g_motor_data.bus_current= (g_motor_data.current[0] + g_motor_data.current[1]);

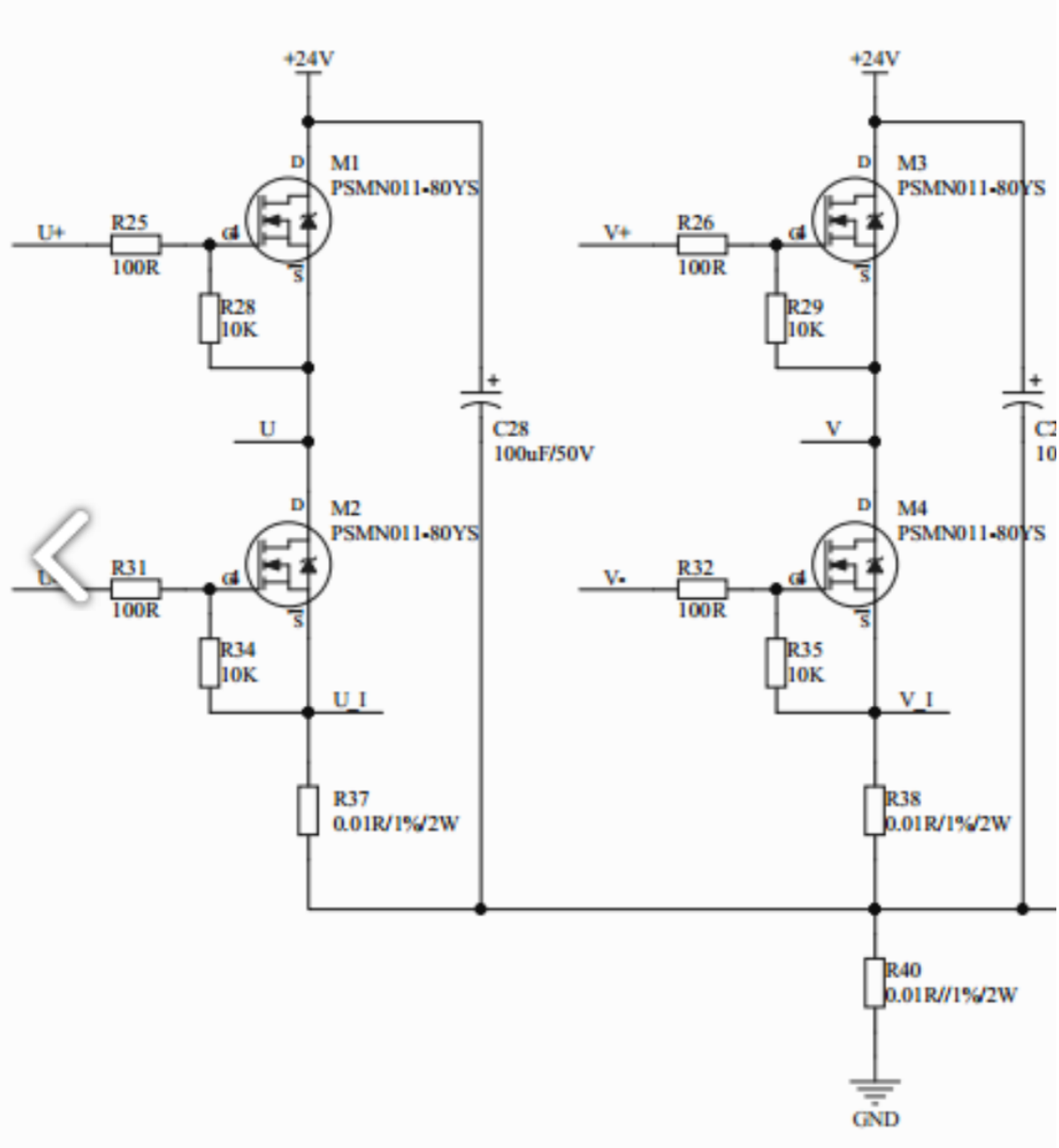
```
  }
else if (g_bldc_motor1.step_sta == 0x01)
  {
    /* UW */
     g_motor_data.bus_current= (g_motor_data.current[0] + g_motor_data.current[2]);
  }
else if (g_bldc_motor1.step_sta == 0x03)
  {
    /* VW */
     g_motor_data.bus_current= (g_motor_data.current[1] + g_motor_data.current[2]);
  }
 else if (g_bldc_motor1.step_sta == 0x02)
  {
    /* UV*/
     g_motor_data.bus_current= (g_motor_data.current[0] + g_motor_data.current[1]);
  }
else if (g_bldc_motor1.step_sta == 0x06)
  {
    /* WU*/
     g_motor_data.bus_current= (g_motor_data.current[0] + g_motor_data.current[2]);
  }
else if (g_bldc_motor1.step_sta == 0x04)
  {
    /* WV*/
     g_motor_data.bus_current= (g_motor_data.current[2] + g_motor_data.current[1]);
  }
```

电流采样分析

在六步换向中，母线电流等于导通的两相电流和原因是

在中断那一个时刻电机电感上有续流电流，假设U+，V-导通，那么在换向时刻，U-始终不是导通的，但是有电感续流，UV方向始终有电流，所以需要采样V相，然后又因为mos管可以反向流过电流所以U-也是有电流，但是此时电流是负值

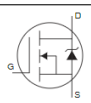
其中等效电路图



因为mos管可以反向流过电流，此时U-电流是负值，母线电流等于这两个直接相加

NMOS具体参数，可以看Is

Diode Characteristics

Symbol	Parameter	Min.	Typ.	Max.	Units	Conditions
I_S	Continuous Source Current (Body Diode)	----	----	80①	A	MOSFET symbol showing the integral reverse p-n junction diode. 
I_{SM}	Pulsed Source Current (Body Diode) ②	----	----	310		
V_{SD}	Diode Forward Voltage	----	----	1.3	V	$T_J = 25°C$, $I_S = 46A$, $V_{GS} = 0V$ ⑤
dv/dt	Peak Diode Recovery	----	27	----	V/ns	$T_J = 175°C$, $I_S = 46A$, $V_{DS} = 75V$ ④
t_{rr}	Reverse Recovery Time	----	33	50	ns	$T_J = 25°C$ $V_R = 64V$,
		----	39	59		$T_J = 125°C$ $I_F = 46A$
Q_{rr}	Reverse Recovery Charge	----	32	48	nC	$T_J = 25°C$ di/dt = 100A/μs ⑤
		----	47	71		$T_J = 125°C$
I_{RRM}	Reverse Recovery Current	----	1.9	----	A	$T_J = 25°C$
t_{on}	Forward Turn-On Time	Intrinsic turn-on time is negligible (turn-on is dominated by LS+LD)				